

데이터프레임 탐색과 선택

describe · 열 선택 · loc과 iloc

전체가 아니라 필요한 부분만 정확히 꺼내는 것 — 그것이 분석의 시작이다

12_02 통계 미리보기 · 13_01 행·열 선택 · 13_02 조건 필터링 도입

핵심 키워드

`.describe()` · 분위수 · 75% vs max · 첫 탐색 4단계

`df['형체력']` · `df[['형체력', '실린더압력']]` · Series · DataFrame

`.columns` · `.index` · `df.loc[행, 열]` · `df.iloc[행, 열]` · 끝 포함 vs 끝 제외 · Boolean Series

2026년 8월 13일 (목) · 18일차 · 45~46차시

국립금오공과대학교 컴퓨터공학부 박민호

이 책을 읽는 법

벤티 · 인덱서라는 새 분류 · 지난 회차와의 연결

1. 항목 벤티 — 오늘 새로 생긴 분류 하나

오늘은 지금까지 없던 분류가 하나 등장한다. **인덱서(indexer)** 다. `loc`과 `iloc`은 메서드도 속성도 아니다. 뒤에 소괄호가 아니라 대괄호가 붙는다. `df.loc(0)`이라고 쓰면 오류다. 이 구분을 처음부터 잡아 두자.

벤티	판별법	이번 회차 예
메서드	점 + 이름 + 소괄호 ()	<code>.head()</code> · <code>.info()</code> · <code>.describe()</code> · <code>.mean()</code>
속성	점 + 이름, 괄호 없음	<code>.shape</code> · <code>.columns</code> · <code>.index</code> · <code>.dtypes</code>
패턴	점 + 이름 + 대괄호 [] ← 인덱서	<code>.loc[0]</code> · <code>.iloc[0:2, 1:3]</code>
문법	표기 규칙 자체	<code>df['형체력']</code> · <code>df[['형체력', '주조압력']]</code>
자료형	값의 종류	<code>Series</code> · <code>DataFrame</code> · <code>int64</code> · <code>object</code>
개념	문법이 아닌 사고방식	라벨과 위치 · 첫 탐색 · API

개념

3초 판별법이 한 단계 늘었다. ① 앞에 점이 있나? 없으면 함수. ② 점이 있고 뒤가 소괄호면 메서드. ③ 점이 있고 괄호가 없으면 속성. ④ 점이 있고 뒤가 대괄호면 인덱서. 오늘 `loc`·`iloc`이 네 번째에 해당한다.

2. 이 책이 다루는 범위 — 그리고 빠져 있는 하루

솔직하게 밝혀 둔다. **8월 12일(17일차) 정리본이 아직 없다.** 그날 배운 Pandas 입문(`read_csv`)과 DataFrame 구조 파악(`head`·`tail`·`shape`·`columns`·`index`·`dtypes`·`info`)이 문서로 정리되어 있지 않다. 오늘 수업은 그 다음 순서인 `describe`부터 시작한다.

그래서 이 책의 **CHAPTER 0**에 **지난 시간 압축 복습**을 넣었다. 12일 정리본이 없어도 이 책만으로 오늘 내용을 따라갈 수 있게 하기 위해서다. 다만 압축본이라 그날의 실습과 세부 설명은 빠져 있다.

회차	날짜	교안	정리본
16일차	8/11 (화)	10_02 넘파이 연산·통계	있음

회차	날짜	교안	정리본
17일차	8/12 (수)	12_01 Pandas 입문 + 12_02 PART 01	없음 — STT 미확보
18일차	8/13 (목)	12_02 PART 02 + 13_01 + 13_02 도입	이 책

주의

이번 회차 자료 결손 고지. ① 위 표의 17일차 공백. ② 오늘분 12_02 강사 코드가 없다 — 올려받은 12_combined.py는 파일명이 전부 42_260812로 어제 것이고, head·tail·shape·info 실습(1~5)까지만 들어 있다. describe 실습(6~9) 코드가 없어 교안 p41~p63과 전사본으로 복원했다. ③ 강사 코드 13_combined.py 실습 5에 오타가 있어 실행이 멈춘다(부록 B-3 참고).

3. 지난 회차와의 연결 — 넘파이가 판다스 안에서 계속 돈다

16일차에 "Pandas의 내부는 전부 넘파이 배열"이라고 예고했다. 오늘 그 말이 실제로 확인된다. 강사가 13_02 도입에서 짚은 문장이 정확하다 — "판다스의 새로운 문법이 아니라 넘파이에 있던 기법을 그대로 가져와 쓰는 것" 이다.

넘파이 (15·16일차)	판다스 (오늘)	같은 점 · 다른 점
arr.shape · arr.dtype	df.shape · df.dtypes	속성인 것까지 동일 · 열마다 자료형이 다름
data[:, 0] 번호로 열	df['형체력'] 이름으로 열	번호 → 이름으로 바뀜
arr.mean() · arr.std()	df.describe()	8개 통계를 한 번에
a[1:4] 끝 제외	df.iloc[1:4] 끝 제외	동일 — 그런데 loc은 끝 포함
v > 85 불리언 배열	df['비스킷두께'] >= 13	Boolean Series — 원리 동일
v[v > 85]	df[df['비스킷두께'] >= 13]	조건 필터링 문법 거의 같음

CONTENTS

목차

오늘 배운 것 전체 지도

장 · STEP	무엇을 배우나	핵심
CHAPTER 0	오늘의 지도 — 지난 시간 압축 복습 · API · 인덱서	head · shape · info 복습
STEP 1	describe — 숫자 열 통계를 한 번에	.describe() · 8개 통계량
STEP 2	통계로 이상 신호 감 잡기 — 75%와 max의 거리	의심일 뿐 확정 아님
STEP 3	첫 탐색 4단계 체크리스트	head→shape→info→describe
STEP 4	열 선택 — 대괄호 한 겹과 두 겹	df['A'] vs df[['A']]
STEP 5	인덱스 — 라벨과 위치는 어긋날 수 있다	정렬하면 갈라진다
STEP 6	loc — 이름표로 고르기	df.loc[행, 열] · 끝 포함
STEP 7	iloc — 순서로 고르기	df.iloc[행, 열] · 끝 제외
STEP 8	loc vs iloc — 오늘 최다 실수 구간	0:10이 11줄인 함정
STEP 9	다음 시간 맛보기 — Boolean Series	df[df['두께'] >= 13]
부록 A	치트시트 — 오늘 나온 전부 요약	한 장 요약
부록 B	오류 · 함정 사전 + 제출 코드 교정 + 원자료 오탈자	✗ → ✓
부록 C	실습 1~6 하이라이트 · 보충답안 · 다음 시간 예고	조건 필터링 · 정렬

오늘 하루의 흐름 — 탐색에서 선택으로

오늘은 성격이 다른 두 덩어리가 이어졌다. 오전은 "이 데이터가 어떻게 생겼나" 를 마무리하는 탐색 단계(12_02), 오후는 "거기서 필요한 부분만 어떻게 꺼내나" 를 배우는 선택 단계(13_01)다. 그리고 마지막에 다음 시간 주제인 조건 필터링을 살짝 열어 봤다.



설비 적용

오늘 두 종류의 실데이터를 썼다. 오전은 **지하철 공기압축기(MetroPT-3)** — 전동차 브레이크·출입문용 압축공기를 만드는 설비로 압력·오일온도·모터전류를 1초마다 기록한다. 오후는 **주조 다이캐스팅** — 용융 금속을 금형에 고압 주입해 부품을 성형하는 설비로, 한 번의 주입(샷)마다 실린더압력·주조압력·사이클타임·비스킷두께·형체력·품질등급을 남긴다. 오후 데이터는 **품질등급(양품/주의/불량)**이라는 **정답 열**이 있다는 점이 특별하다 — 나중에 고장 예측 모델을 학습시킬 때 이 열이 정답지가 된다.

오늘의 지도

지난 시간 압축 복습 · API라는 말 · 인덱서

0-1. 지난 시간 압축 복습 — Pandas 첫 세 시간

오늘 수업은 어제 내용을 전제로 시작한다. 17일차 정리가 없으니 여기서 압축해 짚는다. **이 표만 이해하면 오늘 내용을 따라갈 수 있다.**

도구	형태	하는 일	기억할 점
<code>pd.read_csv()</code>	함수	CSV를 DataFrame으로 읽기	<code>sep</code> · <code>encoding</code> · <code>usecols</code> · <code>nrows</code>
<code>.head(n)</code>	메서드	앞에서 n줄 (기본 5줄)	데이터보다 큰 수를 넣어도 오류 없음
<code>.tail(n)</code>	메서드	뒤에서 n줄	head와 짝으로 — 양 끝이 멀쩡하면 가운데도 대체로 신뢰
<code>.shape</code>	속성	(행, 열) 튜플	괄호 없음 · 앞이 행 뒤가 열
<code>.columns</code>	속성	열 이름 목록	복사해 쓰면 오타 방지
<code>.index</code>	속성	행 이름표 범위	0부터 시작 — 200행이면 0~199
<code>.dtypes</code>	속성	열별 자료형	<code>int64</code> · <code>float64</code> · <code>object</code>
<code>.info()</code>	메서드	구조 + 결측 종합	Non-Null Count가 핵심

어제 배운 것 한 화면 — 여기서 오늘이 시작된다

```
import pandas as pd
```

```
df = pd.read_csv("data/12_metro_compressor.csv")
```

```
print(df.shape)
print(df.columns.tolist())
df.info()
```

- ▶ (200, 7)
- ▶ ['측정시각', '압축압력', '배출압력', '저장압력', '오일온도', '모터전류', '가동상태']
- ▶ RangeIndex: 200 entries, 0 to 199
- ▶ 오일온도 199 non-null float64 ← 200이 아니다 = 결측 1개
- ▶ 모터전류 200 non-null float64

개념

`info()`의 **Non-Null Count**가 핵심이다. 전체가 200인데 오일온도만 199면 **결측이 1개** 있다는 뜻이다. NaN(Not a Number)은 오류가 아니라 "그 시점에 측정값이 없음"을 나타내는 정상 표시다. **숫자 0과 전혀 다르다** — 0은 측정된 값이고 NaN은 측정 자체가 없다.

0-2. API라는 말 — 강사가 첫 20분을 쓴 이유

오늘 수업은 문법이 아니라 **용어 정리**로 시작했다. `df.shape`처럼 괄호가 없는 것도 있고 `df.head()`처럼 괄호가 있는 것도 있는데, 이것들을 뭉뚱그려 부를 말이 필요했기 때문이다. 그 말이 **API**다.

개념 API (Application Programming Interface) 라이브러리가 제공하는 호출 가능한 기능 전체
정의 Pandas가 제공하는 모든 기능 — 함수·메서드·속성·인덱서를 통틀어 부르는 말.

왜 쓰나 `head()`는 메서드, `shape`은 속성, `read_csv()`는 함수다. 형태가 제각각인데 **"판다스가 제공하는 것들"**이라고 묶어 말할 때 API라고 한다. 공식 문서 제목이 「Pandas API reference」인 이유가 이것이다.

응용 실무에서 API는 두 층위로 쓰인다. ① **라이브러리 API** — 지금처럼 내 컴퓨터에 설치된 기능을 부르는 것. ② **웹 API** — 원격 서버에 요청을 보내 답을 받아오는 것. ChatGPT를 코드로 부르는 것이 후자다. 데이터 분석을 하다 보면 두 가지를 다 만나게 되므로, 문맥으로 구분하는 감각이 필요하다.

```
# ① 라이브러리 API — 내 컴퓨터 안의 기능 호출
import pandas as pd
df = pd.read_csv("data.csv")    # pandas API

# ② 웹 API — 원격 서버에 요청 (다음 학습 범위)
# import requests
# r = requests.get("https://api.example.com/sensors")
# df = pd.DataFrame(r.json())
```

개념

강사가 덧붙인 말 — `read_csv` 말고도 `read_excel`, `read_json` 같은 것들이 있다. **CSV만 되는 게 아니다**. 파일 형식이 바뀌어도 함수 이름만 바뀌면 되도록 설계되어 있다.

0-3. 인덱스 — 오늘 새로 생긴 네 번째 분류

지금까지 함수·메서드·속성 세 가지로 충분했다. 오늘 `loc`과 `iloc`이 나오면서 하나가 더 필요해졌다. 이 둘은 **점을 찍고 부르지만 뒤에 대괄호가 온다**.

세 가지 형태가 한 화면에

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")
```

```
# 속성 — 괄호 없음
print(df.shape)
```

```
# 메서드 — 소괄호
print(df.head(2))
```

```
# 인덱서 — 대괄호 (소괄호를 쓰면 오류)
print(df.loc[0, "품질등급"])
# print(df.loc(0)) ← TypeError: not callable
```

```
▶ (30, 7)
▶ 샷 실린더압력  주조압력  사이클타임  비스킷두께  형체력  품질등급
▶ 0   1  214.0  1037.0    20.7    10.0   258.0   양품
▶ 1   2  217.0  1052.0    20.7    11.0   257.0   양품
▶ 양품
```

질문	판정	예
① 앞에 점(.)이 없다	함수 모듈 이름이 앞에	<code>pd.read_csv()</code>
② 점 + 소괄호 ()	메서드 동작을 시킴	<code>df.head() · df.describe()</code>
③ 점 + 괄호 없음	속성 이미 가진 정보	<code>df.shape · df.columns</code>
④ 점 + 대괄호 []	패턴 인덱서 — 위치를 줌	<code>df.loc[0] · df.iloc[0:2]</code>

자주 하는 실수

`df.loc(0)`처럼 소괄호를 쓰면 `TypeError: _LocIndexer object is not callable`이 난다. 반대로 열을 고를 때 `df.loc`을 빼먹고 `df[0]`이라고 쓰면 "0이라는 이름의 열"을 찾다가 `KeyError`가 난다. **이름만 대괄호에 넣으면 열, loc/iloc이 앞에 붙으면 행이다.**

0-4. 오늘 나온 이름 전체

분류	이름	한 줄
메서드	<code>.describe()</code>	숫자 열의 8개 통계를 한 번에
속성	<code>.columns</code>	열 이름 목록 — 고르기 전에 확인

분류	이름	한 줄
속성	<code>.index</code>	행 이름표 범위 — 0부터
자료형	Series	1차원 — 열 하나를 꺼낸 결과
자료형	DataFrame	2차원 표 — 대괄호 두 겹의 결과
문법	<code>df['형체력']</code>	대괄호 한 겹 → Series
문법	<code>df[['형체력', '실린더압력']]</code>	대괄호 두 겹 → DataFrame
패턴	<code>df.loc[라벨]</code>	이름표로 행 선택 · 끝 포함
패턴	<code>df.loc[행, 열]</code>	행과 열 동시 — 쉼표 앞이 행
패턴	<code>df.iloc[위치]</code>	순서 번호로 행 선택 · 끝 제외
패턴	<code>df.iloc[행, 열]</code>	열도 0부터 세는 번호
개념	라벨 vs 위치	정렬하면 둘이 갈라진다
개념	Boolean Series	조건 결과가 참·거짓 시리즈
개념	API	라이브러리가 제공하는 기능 전체

STEP 1

describe

숫자 열 통계를 한 번에

`info()`까지는 구조를 봤다. 몇 행 몇 열인지, 자료형이 무엇인지, 결측이 있는지. 그런데 구조가 멀쩡해도 값 자체가 이상할 수 있다. 온도가 전부 숫자로 잘 들어왔는데 그중 하나가 100도가 넘는다면? 구조 점검으로는 못 잡는다. 그래서 `describe()`가 필요하다.

1-1. describe — 한 줄로 끝나는 통계 요약

메서드 `.describe()` 숫자 열들의 기초 통계를 한 번에 계산

정의 DataFrame의 숫자 열만 골라 개수·평균·표준편차·최소·분위수·최대 여덟 가지를 자동으로 계산해 표로 돌려준다.

형태 `df.describe()`

```
import pandas as pd
pd.set_option("display.width", 200)

df = pd.read_csv("data/12_metro_compressor.csv")
print(df.describe().round(2))
```

```
▶   압축압력  배출압력  저장압력  오일온도  모터전류
▶ count    200.00    200.00    200.00    199.00    200.00
▶ mean       9.17     -0.02       9.17     63.18       2.06
▶ std        0.58      0.05      0.58      6.25       2.20
▶ min        8.13     -0.03      8.13     50.10      0.03
▶ 25%        8.70     -0.02      8.70     58.10      0.04
▶ 50%        9.18     -0.02      9.18     62.90      0.04
▶ 75%        9.66     -0.02      9.66     68.10      3.81
▶ max       10.22      0.60     10.22     75.00      6.19
```

왜 쓰나 수만 줄을 다 보지 않고도 전체 모습을 한 화면에 파악한다. 강사 표현대로 "매번 데이터 제대로 들어왔는지 확인하려면 이거 한 줄이면 끝난다".

응용 `describe()`는 글자 열을 자동으로 뺀다. 위 출력에 측정시각과 가동상태가 없는 이유다. 글자 열까지 보려면 `include="all"`을 준다. 그러면 숫자 열에 없던 `unique`(고유값 개수)·`top`(최빈값)·`freq`(최빈값 빈도)가 함께 나온다.

```
# 글자 열까지 포함
print(df.describe(include="all").round(2))

# 글자 열만
print(df.describe(include="object"))

# 특정 열만
print(df[["오일온도", "모터전류"]].describe().round(2))
```

```
▶      측정시각  압축압력  ...  가동상태
▶ count              200    200.0  ...    200
▶ unique              200     NaN  ...     2
▶ top    2020-02-27 06:38:47    NaN  ...    정지
▶ freq              1      NaN  ...    104
```

개념

count가 199인 열(오일온도)이 눈에 띈다. 나머지는 200인데 혼자 199 — info()의 Non-Null Count와 같은 정보다. describe() 한 줄로 결측까지 함께 잡힌다.

1-2. 8개 통계량 — 개수와 중심

통계량	뜻	읽는 법	예 (오일온도)
count	빈 칸 아닌 값의 개수	전체 행 수보다 작으면 결측 있음	199 (전체 200 → 결측 1)
mean	평균 — 대표값	"보통 이 정도"	63.18°C
std	표준편차 — 흩어진 정도	작으면 안정, 크면 불안정	6.25°C

1-3. 8개 통계량 — 위치

통계량	뜻	읽는 법	예 (오일온도)
min	가장 작은 값	정상 하한 감각	50.10°C
25%	아래에서 1/4 지점	하위 4분의 1 경계	58.10°C
50%	한가운데 값 = 중앙값	평균과 비교해 이상치 점검	62.90°C
75%	위에서 1/4 지점	상위 4분의 1 경계	68.10°C
max	가장 큰 값	75%와의 거리가 핵심	75.00°C

여덟 개를 다 외울 필요는 없다. 교안도 "여덟 개 중 mean·min·max 셋부터 읽으면 충분" 이라고 못 박는다. 익숙해지면 50%(중앙값)와 75%를 추가로 보고, 마지막에 std를 본다.

describe 결과에서 값 하나만 꺼내기

```
# 특정 통계량만 꺼내기 — describe 결과도 DataFrame이다
de = df.describe()
```

```
print("오일온도 평균 :", round(de.loc["mean", "오일온도"], 2))
print("오일온도 중앙값:", round(de.loc["50%", "오일온도"], 2))
print("오일온도 최대 :", round(de.loc["max", "오일온도"], 2))
```

```
# 개별 메서드로도 같은 값
```

```
print("검산:", round(df["오일온도"].mean(), 2), round(df["오일온도"].max(), 2))
```

```
▶ 오일온도 평균 : 63.18
▶ 오일온도 중앙값: 62.9
▶ 오일온도 최대 : 75.0
▶ 검산: 63.18 75.0
```

연결

de.loc["mean", "오일온도"]가 눈에 익지 않을 것이다. **오늘 오후에 배울 loc 이다.** describe()의 결과도 DataFrame이므로 행 이름(mean·50%)과 열 이름으로 값 하나를 짚어낼 수 있다. STEP 6에서 제대로 다룬다.

1-4. 평균과 표준편차는 짝으로 읽는다

16일차 넘파이에서 배운 내용이 그대로 돌아온다. **평균이 같아도 표준편차가 다르면 전혀 다른 데이터다.** 교안 p44가 이 질문을 던진다 — "평균이 같으면 같은 데이터인가?"

평균만 보면 똑같다

```
import pandas as pd
```

```
# 평균은 같고 표준편차만 다른 두 설비
```

```
A = pd.Series([74, 75, 76, 75, 75]) # 안정
```

```
B = pd.Series([50, 100, 60, 90, 75]) # 불안정
```

```
for name, s in [("설비 A", A), ("설비 B", B)]:
```

```
    print(f"{name}: 평균 {s.mean():.1f}도 · 표준편차 {s.std():.1f}도 "
```

```
          f"· 범위 {s.min():.0f}~{s.max():.0f}도")
```

▶ 설비 A: 평균 75.0도 · 표준편차 0.7도 · 범위 74~76도

▶ 설비 B: 평균 75.0도 · 표준편차 20.6도 · 범위 50~100도

설비 적용

두 설비 모두 평균 75도다. 그런데 A는 74~76도에서 조용히 돌고 B는 50~100도를 오간다. **표준편차 0.7 대 20.6**

— 이 차이를 못 보면 B의 고장 징후를 놓친다. 평소 안정적이던 값이 흔들리기 시작하면 그것이 주의 신호다. 압축기 데이터의 오일온도 표준편차 6.25는 평균 63의 10% 수준으로 비교적 안정적이다.

주의

Pandas의 `.std()`는 기본이 **표본 표준편차**(`ddof=1`) 이고, **넘파이**의 `.std()`는 기본이 **모집단 표준편차**(`ddof=0`) 다. 그래서 같은 데이터인데 두 라이브러리 값이 미세하게 다를 수 있다. 16일차 부록에서 예고한 그 차이가 여기서 실제로 나타난다.

STEP 2

통계로 이상 신호 감 잡기

75%와 max 사이의 거리

`describe()`를 뽑았다고 끝이 아니다. 숫자를 읽을 줄 알아야 한다. 교안이 제시하는 기준은 딱 하나다 — **75% 지점과 최댓값이 얼마나 떨어져 있는가.**

2-1. 왜 75%와 max인가

데이터의 4분의 3이 75% 지점 아래에 있다. 그러니 **75%와 max가 가까우면** 나머지 4분의 1도 그 근처에 몰려 있다는 뜻 — 고른 분포다. 반대로 **멀면** 대부분은 정상인데 일부가 멀리 튀었다는 강한 신호다.

75%와 max 거리 자동 계산

```
import pandas as pd
```

```
df = pd.read_csv("data/12_metro_compressor.csv")
de = df.describe()
```

```
for c in de.columns:
    q75, mx = de.loc["75%", c], de.loc["max", c]
    gap = mx - q75
    print(f"{c:6s} 75%={q75:8.2f} max={mx:8.2f} 차이={gap:7.2f}")
```

▶ 압축압력	75%=	9.66	max=	10.22	차이=	0.56
▶ 배출압력	75%=	-0.02	max=	0.60	차이=	0.62
▶ 저장압력	75%=	9.66	max=	10.22	차이=	0.56
▶ 오일온도	75%=	68.10	max=	75.00	차이=	6.90
▶ 모터전류	75%=	3.81	max=	6.19	차이=	2.38

차이의 절대값만 보면 오일온도(6.90)가 가장 크다. 하지만 **단위가 다르므로 절대값 비교는 위험하다**. 압력은 10 근처, 온도는 63 근처의 값이다. 비율로 보는 편이 정확하다.

중앙값 대비 비율로 다시 보기

```
for c in de.columns:
    q75, mx, med = de.loc["75%", c], de.loc["max", c], de.loc["50%", c]
    if med == 0:
        continue
    ratio = (mx - q75) / abs(med) * 100
    flag = "▲ 이상 의심" if ratio > 30 else "정상 범위"
    print(f"{c:6s} 중앙값 대비 {ratio:6.1f}% {flag}")
```

- ▶ 압축압력 중앙값 대비 6.0% 정상 범위
- ▶ 배출압력 중앙값 대비 3100.0% ▲ 이상 의심
- ▶ 저장압력 중앙값 대비 6.2% 정상 범위
- ▶ 오일온도 중앙값 대비 11.0% 정상 범위
- ▶ 모터전류 중앙값 대비 5943.8% ▲ 이상 의심

설비 적용

모터전류가 걸렸다. 중앙값 0.04인데 max가 6.19다. 압축기가 꺼져 있을 때는 전류가 거의 0이고 가동 중에는 급격히 올라가기 때문이다. 통계적으로는 이상해 보이지만 실제로는 정상 동작이다. 배출압력도 값이 -0.02 근처라 비율 계산이 무너진다. 통계는 수상한 곳을 찾아 주는 출발점일 뿐, 확정은 현장 지식이 한다.

2-2. 중앙값과 평균의 관계로 분포 모양 읽기

두 번째 읽기 기법이다. 중앙값(50%)과 평균이 비슷하면 고른 분포, 크게 다르면 한쪽으로 치우쳤다는 뜻이다. 16일차에 배운 「데이터 건강 점검」과 같은 원리인데, describe() 한 줄이면 두 값이 나란히 나오니 훨씬 편하다.

관계	뜻	설비 데이터에서
평균 ≈ 중앙값	좌우 대칭에 가까운 고른 분포	안정적으로 돌고 있음
평균 > 중앙값	위쪽에 멀리 튕 값이 있음	과열·과부하 순간 존재 의심
평균 < 중앙값	아래쪽에 멀리 튕 값이 있음	정지·급락 순간 존재 의심

평균 vs 중앙값 — 분포 모양 판정

```
for c in de.columns:
    mean, med = de.loc["mean", c], de.loc["50%", c]
    diff = mean - med
    shape = "위로 치우침" if diff > 0.01 else ("아래로 치우침" if diff < -0.01 else
"고름")
    print(f"{c:6s} 평균={mean:8.2f} 중앙값={med:8.2f} → {shape}")
```

```
▶ 압축압력 평균=    9.17 중앙값=    9.18 → 고름
▶ 배출압력 평균=   -0.02 중앙값=   -0.02 → 고름
▶ 저장압력 평균=    9.17 중앙값=    9.18 → 고름
▶ 오일온도 평균=   63.18 중앙값=   62.90 → 위로 치우침
▶ 모터전류 평균=    2.06 중앙값=    0.04 → 위로 치우침
```

2-3. 의심일 뿐 확정이 아니다

교안 p49가 이 장의 결론을 못 박는다 — "describe는 수상한 곳을 찾는 출발점 (확정은 추가 확인)". **최댓값이 크다고 무조건 오류는 아니다.** 실제로 과열이었을 수도 있다.

개념

강사가 든 예가 좋았다. 온도 데이터를 받았는데 **100도가 넘는 값**이 있으면 당황스럽다. 그런데 그 데이터가 **미국 자료라 화씨(°F)** 였다면? 100°F는 섭씨 38도로 지극히 정상이다. Kaggle 같은 해외 데이터셋을 쓸 때 실제로 자주 벌어지는 일이다. **통계로 이상을 발견하면 다음 질문은 "왜 그런가"** 이지 "제거하자"가 아니다.

이상해 보이는 신호	기계 오류일 수도	정상일 수도
최댓값이 유난히 큼	센서 오작동 · 통신 오류	실제 과열 · 과부하 순간
중앙값이 0에 가까운데 max가 큼	값이 잘못 기록됨	가동/정지가 섞인 데이터
단위가 이상함 (온도 100 초과)	자료형 오인식	화씨 · 켈빈 등 다른 단위
특정 값만 계속 반복	센서 고착(stuck)	실제로 일정하게 제어 중

2-4. 실습 6·7 — describe 해석하고 문장으로 쓰기

교안 실습 6은 "75%와 max 차이가 큰 열을 두 개 이상 찾기", 실습 7은 "통계를 자기 말로 풀어 설명하기" 다. 실습 7이 특히 중요하다 — 숫자를 옮겨 적는 것과 의미를 설명하는 것은 전혀 다른 능력이고, 실무 보고서는 후자를 요구한다.

실습 7 — 통계를 문장으로

```
df = pd.read_csv("data/12_metro_compressor.csv")
de = df.describe()

col = "오일온도"
print(f"[{col}] 통계 묘사")
print(f"이 값은 보통 {de.loc['mean', col]:.1f}°C 정도이고,")
print(f"가장 낮을 때 {de.loc['min', col]:.1f}°C, 높을 때 {de.loc['max', col]:.1f}°C까지 올라간다.")
print(f"한가운데 값은 {de.loc['50%', col]:.1f}°C로 평균과 비슷해 대체로 고른 분포다.")
print(f"표준편차는 {de.loc['std', col]:.1f}°C로 평균의 약 {de.loc['std', col]/de.loc['mean', col]*100:.0f}% 수준 — 비교적 안정적이다.")
```

-
- ▶ [오일온도] 통계 묘사
 - ▶ 이 값은 보통 63.2°C 정도이고,
 - ▶ 가장 낮을 때 50.1°C, 높을 때 75.0°C까지 올라간다.
 - ▶ 한가운데 값은 62.9°C로 평균과 비슷해 대체로 고른 분포다.
 - ▶ 표준편차는 6.2°C로 평균의 약 10% 수준 — 비교적 안정적이다.

주의

실습 7의 체크포인트는 "숫자를 그냥 옮기지 않고 의미로 풀어 썼는가" 다. "평균 63.18, 최대 75.0"은 숫자 나열이고, "보통 63도 근처에서 돌다가 최대 75도까지 올라간다 — 표준편차가 평균의 10% 수준이라 비교적 안정적"은 해석이다. 후자를 쓸 수 있어야 분석가다.

STEP 3

첫 탐색 4단계

head → shape → info → describe

여기까지가 「데이터 첫 탐색」의 전부다. 교안 p54가 이것을 네 단계 체크리스트로 묶는다. 이 네 가지만 거치면 어떤 데이터를 받아도 막막하지 않다.



단계	도구	던지는 질문	무엇을 얻나
STEP 1	<code>df.head()</code>	무엇에 관한 데이터인가	열 이름 · 실제 값 · 큰 그림
STEP 2	<code>df.shape</code>	얼마나 큰가	행 수 · 열 수
STEP 3	<code>df.info()</code>	어디가 문제인가	자료형 · Non-Null(결측)
STEP 4	<code>df.describe()</code>	값이 정상인가	평균 · 분위수 · 이상 신호

네 단계를 함수 하나로 — 새 데이터마다 재사용

```
import pandas as pd
```

```
def explore(path, name):
```

```
    """첫 탐색 4단계를 한 번에"""
```

```
    df = pd.read_csv(path)
```

```
    print(f"===== {name} =====")
```

```
    print(f"[1] head\n{df.head(3)}\n")
```

```
    print(f"[2] shape: {df.shape}")
```

```
    print(f"[3] 결측 합계: {df.isna().sum().sum()}개")
```

```
    print(f"[4] 숫자 열 {df.describe().shape[1]}개 통계 산출 완료\n")
```

```
    return df
```

```
df_c = explore("data/12_metro_compressor.csv", "압축기")
```

```
df_d = explore("data/12_metro_digital.csv", "디지털 신호")
```

```
▶ ===== 압축기 =====
```

```
▶ [2] shape: (200, 7)
```

```
▶ [3] 결측 합계: 1개
```

```
▶ [4] 숫자 열 5개 통계 산출 완료
```

```
▶ ===== 디지털 신호 =====
```

```
▶ [2] shape: (120, 4)
```

```
▶ [3] 결측 합계: 0개
```

```
▶ [4] 숫자 열 3개 통계 산출 완료
```

응용

위처럼 함수로 묶어 두면 새 데이터를 받을 때마다 한 줄로 끝난다. 11일차에 배운 def가 여기서 쓰인다. 실무에서는 이런 탐색 함수를 개인 유틸리티 파일에 모아 두고 프로젝트마다 가져다 쓴다. 포트폴리오 레포에 `utils/explore.py` 같은 파일이 있으면 "반복 작업을 자동화할 줄 안다"는 신호가 된다.

3-1. 깔끔한 데이터와 지저분한 데이터

교안 실습 8은 성격이 정반대인 두 데이터를 나란히 놓고 같은 체크리스트를 적용해 보라고 한다. 이 비교가 첫 탐색이 왜 필요한지를 몸으로 알려 준다.

구분	압축기 (CLEAN)	디지털 신호 (MESSY)
성격	교육용 인공 데이터	실제 공정 데이터
결측	거의 없음 (1개)	열마다 제각각 — 많은 경우 수백 개
값 범위	열마다 일관됨	제각각 — 정리 필요

구분	압축기 (CLEAN)	디지털 신호 (MESSY)
바로 분석	가능	불가 — 전처리 먼저
현실은	교과서 같은 예외	대부분 이쪽

개념

교안의 교훈은 분명하다 — **"현실 데이터는 대부분 디지털 신호 쪽"** 이다. 깔끔한 데이터로만 연습하면 실무에서 무너진다. 그래서 첫 탐색으로 **"이 데이터를 바로 쓸 수 있나, 손봐야 하나"** 를 먼저 판단하는 습관이 필요하다. 그 판단을 건너뛰면 결측이 섞인 채로 평균을 내고, 틀린 결론에 도달한다.

3-2. 첫 탐색 리포트 — 분석가의 첫 보고서

교안 종합 실습 3은 탐색 결과를 **여섯 항목의 리포트**로 정리하라고 한다. 실무 분석가가 프로젝트를 시작할 때 실제로 쓰는 문서 형식이다.

항목	무엇을 쓰나	근거 도구
① 개요	어떤 설비의 무슨 데이터인가 · 기간	<code>head()</code>
② 열 구성	열이 몇 개이고 각각 무엇인가	<code>columns</code> · <code>dtypes</code>
③ 결측	어느 열에 몇 개가 비었나	<code>info()</code>
④ 통계	각 열의 평균 · 범위 · 흩어짐	<code>describe()</code>
⑤ 이상 신호	수상한 결과 그 근거	75% vs max · 평균 vs 중앙값
⑥ 종합 의견	바로 분석 가능한가, 정리가 필요한가	위 다섯 항목 종합

설비 적용

실무에서 이 리포트가 첫 미팅 자료가 된다. **"데이터를 받았는데 이런 상태입니다"** 를 한 장으로 정리해 가면, 현장 담당자가 "그 열은 원래 그렇습니다" 같은 결정적 정보를 준다. 통계로 찾은 의심을 현장 지식으로 확정하는 자리다. 리포트 없이 가면 그 대화가 시작되지 않는다.

STEP 4

열 선택

대괄호 한 겹과 두 겹 — 결과 형태가 같린다

여기서부터 교안 13_01이다. 데이터가 어떻게 생겼는지 알았으니, 이제 **필요한 부분만 꺼낼** 차례다. 교안의 첫 문장이 이 단원의 전부를 요약한다 — "전체가 아니라 필요한 부분만 정확히 꺼내는 것이 분석의 시작."

4-1. 왜 손이 아니라 코드로 고르는가

엑셀 필터로도 같은 일을 할 수 있다. 그런데 두 가지가 결정적으로 다르다.

비교	엑셀 필터	코드로 선택
대용량	수백만 줄에서 느려짐	한 줄로 순식간
재사용	새 파일마다 다시 클릭	같은 코드를 그대로 적용
기록	무엇을 걸렀는지 안 남음	코드가 곧 기록 — 재현 가능
자동화	불가	스케줄러에 걸면 매일 자동

4-2. 단일 열 선택 — 대괄호 한 겹

문법 `df['열이름']` 열 하나를 Series로 꺼내기

정의 DataFrame 뒤에 대괄호를 붙이고 열 이름을 **따옴표로 감싸** 쓴다. 결과는 1차원 Series.

형태 `df['형체력']`

```
import pandas as pd

df = pd.read_csv("data/13_diecasting_small.csv")

s = df["형체력"]
print(type(s).__name__)
print(s.shape)
print(s.head(3))

> Series
> (30,)
> 0    258.0
> 1    257.0
> 2    253.0
> Name: 형체력, dtype: float64
```

왜 쓰나 센서 하나가 열 하나다. "이 설비가 뜨거운가" 같은 질문이 곧 "형체력 열을 보자"로 이어진다. 질문이 명확하면 어떤 열을 볼지도 명확해진다.

응용 Series는 값과 인덱스가 짝을 이룬 똑똑한 목록이다. 왼쪽에 인덱스, 오른쪽에 값이 붙어 나오므로 어느 측정의 값인지 추적할 수 있다. 그리고 Series에는 통계 메서드를 바로 쓸 수 있다.

```
s = df["형체력"]

print("평균:", round(s.mean(), 1))
print("최대:", s.max(), "· 최소:", s.min())
print("최댓값이 나온 위치:", s.idxmax())

# describe도 바로 된다
print(s.describe().round(1).to_string())
```

```
▶ 평균: 267.8
▶ 최대: 359.0 · 최소: 222.0
▶ 최댓값이 나온 위치: 22
```

자주 하는 실수

따옴표를 빼면 안 된다. `df[형체력]`이라고 쓰면 파이썬이 형체력을 변수 이름으로 해석해 `NameError`가 난다. 그리고 띄어쓰기·대소문자가 한 글자만 달라도 `KeyError`다.

4-3. 복수 열 선택 — 대괄호 두 겹

문법 `df[['열1', '열2']]` 여러 열을 DataFrame으로 꺼내기

정의 대괄호 안에 리스트로 묶어 여러 열을 고른다. 결과는 2차원 DataFrame.

형태 `df[['형체력', '실린더압력']]`

```

두열 = df[["형체력", "실린더압력"]]
print(type(두열).__name__)
print(두열.shape)
print(두열.head(3))

```

```

▶ DataFrame
▶ (30, 2)
▶   형체력 실린더압력
▶ 0   258.0   214.0
▶ 1   257.0   217.0
▶ 2   253.0   215.0

```

왜 쓰나 분석에 필요한 센서만 골라 두면 **데이터가 가벼워지고 봐야 할 것에 집중**할 수 있다. 센서가 60개여도 지금 필요한 두세 개만 남긴다.

응용 적은 순서가 그대로 결과의 열 순서가 된다. 원본 순서와 무관하게 내가 보고 싶은 순서로 재배치할 수 있다는 뜻이다. 리포트를 만들 때 유용하다.

```

# 원본 순서: 샷 · 실린더압력 · 주조압력 · 사이클타임 · 비스킷두께 · 형체력 · 품질등급

```

```

print(df[["형체력", "실린더압력"]].head(2))
print()
print(df[["실린더압력", "형체력"]].head(2))    # 순서를 바꾸면
print()
print(df[["품질등급", "형체력"]].head(2))      # 정답 열을 앞으로

```

```

▶   형체력 실린더압력
▶ 0   258.0   214.0
▶ 1   257.0   217.0
▶
▶   실린더압력 형체력
▶ 0   214.0   258.0
▶ 1   217.0   257.0

```

주의

안쪽 대괄호를 빠뜨리면 오류다. `df["형체력", "실린더압력"]`처럼 쓰면 튜플로 해석돼 `KeyError`가 난다. 여러 열은 반드시 두 겹.

4-4. 한 겹과 두 겹 — 오늘의 첫 번째 갈림길

같은 형체력 열인데 대괄호 겹수에 따라 **결과 형태가 달라진다**. 이것이 오늘 오후 첫 번째 핵심이다.

검수 하나로 갈리는 결과

```
df = pd.read_csv("data/13_diecasting_small.csv")
```

```
a = df["형체력"]          # 한 겹
```

```
b = df[["형체력"]]        # 두 겹 (열은 하나뿐)
```

```
c = df[["형체력", "실린더압력"]]  # 두 겹, 두 열
```

```
for name, x in [("한 겹 ", a), ("두 겹(1열) ", b), ("두 겹(2열) ", c)]:  
    print(f"{name} {type(x).__name__:10s} shape={x.shape}")
```

▶ 한 겹 Series shape=(30,)

▶ 두 겹(1열) DataFrame shape=(30, 1)

▶ 두 겹(2열) DataFrame shape=(30, 2)

구분	코드	결과	shape	언제 쓰나
한 겹	df['형체력']	Series 1차원	(30,)	통계 계산 · 조건 걸기
두 겹	df[['형체력']]	DataFrame 2차원	(30, 1)	표 형태 유지가 필요할 때
두 겹 여러 열	df[['형체력', '실린더압력']]	DataFrame	(30, 2)	여러 센서 함께 보기

비유

교안이 준 외우는 법이 좋다 — "**두 겹은 바구니**" 다. 바구니에 하나만 들어 있어도 바구니는 바구니다. 그래서 `df[["형체력"]]`은 열이 하나여도 DataFrame이다. **두 겹은 예외 없이 항상 DataFrame** — 일관된 규칙이라 헷갈릴 여지가 없다.

주의

형태가 같으면 쓸 수 있는 기능도 같린다. `df["형체력"].mean()`은 숫자 하나를 돌려주지만, `df[["형체력"]].mean()`은 Series를 돌려준다. 헷갈리면 `type()`을 찍어 보자 — 백 번 설명보다 확실하다.

형태가 다르면 결과도 다르다

```
print(df["형체력"].mean())      # Series → 숫자 하나
print()
print(df[["형체력"]].mean())    # DataFrame → Series
print()
print(df[["형체력", "실린더압력"]].mean())
```

```
▶ 267.8
▶ 형체력 267.8
▶ dtype: float64
▶ 형체력 267.800000
▶ 실린더압력 219.666667
▶ dtype: float64
```

4-5. columns — 고르기 전에 이름부터 확인

속성 `.columns` 열 이름 목록 확인

정의 DataFrame의 모든 열 이름을 Index 객체로 돌려준다. 괄호 없는 속성.

형태 `df.columns` · `df.columns.tolist()`

```
print(df.columns)
print()
print(df.columns.tolist())
```

```
▶ Index(['샷', '실린더압력', '주조압력', '사이클타임', '비스킷두께', '형체력', '품질등급'],
dtype='object')
▶ ['샷', '실린더압력', '주조압력', '사이클타임', '비스킷두께', '형체력', '품질등급']
```

왜 쓰나 철자 한 글자만 달라도 못 찾는다. 보이지 않는 앞뒤 공백, 대소문자 차이까지 그대로 노출되므로 고르기 전에 한번 훑는 것이 오타 오류의 예방책이다.

응용 출력된 이름을 복사해서 붙여 쓰는 것이 가장 확실하다. 강사도 이 방법을 권했다. 그리고 `tolist()`를 붙이면 순수 파이썬 리스트가 되어 `cols[2]`처럼 번호로 꺼내 쓸 수 있다 — 열 이름이 길거나 한글일 때 유용하다.

```
cols = df.columns.tolist()

print("2번 열 이름:", cols[2])
print("마지막 열 :", cols[-1])

# 숫자 열만 골라내기
num_cols = df.select_dtypes(include="number").columns.tolist()
print("숫자 열:", num_cols)

# 그 열들만 통계
print(df[num_cols].mean().round(1).to_string())
```

- ▶ 2번 열 이름: 주조압력
- ▶ 마지막 열 : 품질등급
- ▶ 숫자 열: ['샷', '실린더압력', '주조압력', '사이클타임', '비스킷두께', '형체력']

개념

새 데이터를 받으면 head · info · columns 세 가지를 묶어 **치는 것이 교안이 원하는 습관이다**. 각각 생김새 · 건강 · 정확한 이름을 알려 준다.

4-6. 열 이름이 말썽일 때

외부 데이터는 열 이름이 지저분한 경우가 많다. **눈에 안 보이는 앞뒤 공백**이 대표적이다. KeyError가 났는데 아무리 봐도 철자가 맞을 때는 십중팔구 이것이다.

보이지 않는 공백이 KeyError를 만든다

```
# 열 이름에 공백이 섞인 상황
t = pd.DataFrame({" 온도 ": [1, 2], "압력": [3, 4]})

print(t.columns.tolist())      # 따옴표 안을 잘 보자
# print(t["온도"])              ← KeyError: 온도

# 해결 — 열 이름 전체에서 공백 제거
t.columns = [c.strip() for c in t.columns]
print(t.columns.tolist())
```

- ▶ [' 온도 ', '압력']
- ▶ ['온도', '압력']

메서드 `.rename()` 열 이름 바꾸기

정의 딕셔너리로 {옛이름: 새이름}을 넘겨 특정 열의 이름만 바꾼다.

형태 `df.rename(columns={"옛이름": "새이름"})`

```
df = pd.read_csv("data/13_diecasting_small.csv")
```

```
df2 = df.rename(columns={"형체력": "clamp_force"})  
print(df2.columns.tolist())
```

```
▶ ['샷', '실린더압력', '주조압력', '사이클타임', '비스킷두께', 'clamp_force', '품질등급']
```

왜 쓰나 한글 열 이름은 오타가 잦고 코드 자동완성도 잘 안 된다. 분석 초반에 **영문으로 통일**해 두면 이후 작업이 훨씬 편해진다. 강사도 "변수명·함수 이름을 한글로 짓지 말라"고 강조했다 — 인코딩 문제가 생길 수 있기 때문이다.

응용 `rename`도 **원본을 바꾸지 않고 새 DataFrame을 돌려준다**. 넘파이의 `reshape`·`astype`과 같은 원칙이다.

결과를 새 이름에 담아야 쓸 수 있다.

```
# 여러 열을 한 번에
```

```
df_en = df.rename(columns={"실린더압력": "cyl_pressure",  
                           "주조압력": "cast_pressure",  
                           "형체력": "clamp_force"})  
print(df_en.columns.tolist())
```

```
▶ ['샷', 'cyl_pressure', 'cast_pressure', '사이클타임', '비스킷두께', 'clamp_force',  
    '품질등급']
```

주의

딕셔너리에 없는 열은 그대로 남는다. 일부만 바뀌도 오류가 나지 않는다.

STEP 5

인덱스

라벨과 위치는 어긋날 수 있다

열을 골랐으니 이제 행이다. 그런데 행을 고르려면 먼저 **인덱스**를 이해해야 한다. 교안이 "오늘 모듈에서 가장 중요한 개념"이라고 표시한 부분이다.

5-1. 인덱스란 무엇인가

속성 `.index` 각 행에 붙은 이름표

정의 DataFrame의 행마다 붙는 이름표이자 주소. 아무것도 지정하지 않으면 0부터 시작하는 자동 번호가 붙는다.

형태 `df.index`

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

print(df.index)
print("첫 행 인덱스:", df.index[0], "· 마지막:", df.index[-1])
```

▶ `RangeIndex(start=0, stop=30, step=1)`
▶ 첫 행 인덱스: 0 · 마지막: 29

왜 쓰나 행을 가리키는 주소이기 때문이다. 표를 합칠 때 어느 행과 어느 행이 짝인지도 인덱스로 판단한다. 인덱스가 어긋나면 데이터가 엉뚱하게 짝지어진다.

응용 인덱스는 꼭 숫자일 필요가 없다. 날짜나 ID를 인덱스로 지정하면 이름으로 바로 행을 부를 수 있다. 시계열 데이터에서 날짜를 인덱스로 두는 것이 표준 패턴이다.

```
# 샷 번호를 인덱스로
df2 = pd.read_csv("data/13_diecasting_small.csv", index_col="샷")
print(df2.index[:5])
print()
print(df2.loc[3])    # 3번 샷을 이름으로 바로 호출
```

```
▶ Index([1, 2, 3, 4, 5], dtype='int64', name='샷')
▶ 실린더압력    215.0
▶ 주조압력    1040.0
▶ 사이클타임    20.7
▶ 비스킷두께    21.0
▶ 형체력    253.0
▶ 품질등급    주의
▶ Name: 3, dtype: object
```

자주 하는 실수

첫 행의 인덱스는 1이 아니라 0이다. 30행이면 0~29다. "1번 행을 보려고 1을 넣었는데 두 번째 행이 나오는" 함정이 여기서 생긴다.

5-2. 라벨과 위치 — 두 가지 가리키는 방법

같은 행을 가리키는 방법이 **두 가지**다. 하나는 그 행에 붙은 **이름표(라벨)**, 다른 하나는 맨 위에서부터 센 **순서(위치)**다. 인덱스가 0, 1, 2로 깔끔할 때는 둘이 똑같아서 구분할 필요를 못 느낀다. **하지만 어긋나는 순간이 반드시 온다.**

구분	라벨 (label)	위치 (position)
무엇인가	그 행에 붙은 이름표 = 인덱스 값	표 안에서 맨 위부터 센 순서
값의 범위	인덱스가 들고 있는 것 (0, 17, 양품 ...)	항상 0부터 시작하는 정수
글자 가능?	가능 — 날짜·ID도 인덱스가 될 수 있음	불가 — 정수만
도구	<code>df.loc[]</code>	<code>df.iloc[]</code>

비유

교안의 아파트 비유가 정확하다. **101호는 이름표(라벨), 1층 둘째 집은 위치**다. 보통은 101호가 1층 첫째지만, 리모델링으로 호수를 다시 매기면 어긋난다. 데이터에서 그 "리모델링"에 해당하는 것이 **정렬(sort)** 이다.

5-3. 정렬하면 갈라진다 — 직접 확인

말로만 들으면 와닿지 않는다. 실제로 정렬해 보면 라벨과 위치가 분리되는 순간이 눈에 보인다.

정렬 한 번에 라벨과 위치가 갈라진다

```
df = pd.read_csv("data/13_diecasting_small.csv")

print("[정렬 전] 맨 위 3줄")
print(df[["형체력"]].head(3))

s = df.sort_values("형체력", ascending=False)
print("\n[정렬 후] 맨 위 3줄 — 왼쪽 인덱스에 주목")
print(s[["형체력"]].head(3))
print("  loc[0]   (라벨 0번) =", s.loc[0, "형체력"])
print("  iloc[0]  (위치 0번) =", s.iloc[0]["형체력"])
```

```
▶ [정렬 전] 맨 위 3줄
▶   형체력
▶ 0   258.0
▶ 1   257.0
▶ 2   253.0
▶
▶ [정렬 후] 맨 위 3줄 — 왼쪽 인덱스에 주목
▶   형체력
▶ 22   359.0
▶ 17   357.0
▶ 27   355.0
▶   loc[0]   (라벨 0번) = 258.0
▶   iloc[0]  (위치 0번) = 359.0
```

정렬 후 맨 위 행의 **인덱스**가 22로 바뀌었다. 이 행을 라벨로 부르면 22번, 위치로 부르면 0번째다. 그래서 `s.loc[0]`은 원래 0번 행(258.0)을, `s.iloc[0]`은 맨 위 행(359.0)을 가져온다.

설비 적용

현장에서 이런 상황은 매일 생긴다. **"형체력이 가장 낮은 상위 5개 샷을 보자"**는 곧 정렬이다. 정렬한 뒤 `loc[0:4]`를 쓰면 엉뚱한 샷 다섯 개가 나온다 — 원래 인덱스 0~4번을 가져오기 때문이다. **정렬 후에는 `iloc`을 써야 진짜 상위 5개가 나온다. 오늘 배우는 구분이 실무에서 바로 쓰이는 지점이다.**

자주 하는 실수

가장 무서운 건 **오류가 안 난다**는 것이다. `s.loc[0]`도 멀쩡히 값을 돌려주고 코드도 정상 종료된다. 결과가 그럴듯해서 틀린 줄도 모른다. **정렬·필터링 뒤에 행을 고를 때는 `loc`인지 `iloc`인지 반드시 의식하자.**

STEP 6

loc

이름표로 고르기 — 끝까지 포함한다

6-1. loc 단일 행 선택

패턴 `df.loc[라벨]` 이름표로 행 하나 선택

정의 `loc` 뒤 대괄호에 **인덱스 라벨**을 넣어 그 행 전체를 가져온다. 결과는 `Series`.

형태 `df.loc[라벨]`

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

row = df.loc[0]
print(type(row).__name__)
print(row)
```

- ▶ Series
- ▶ 샷 1
- ▶ 실린더압력 214.0
- ▶ 주조압력 1037.0
- ▶ 사이클타임 20.7
- ▶ 비스킷두께 10.0
- ▶ 형체력 258.0
- ▶ 품질등급 양품
- ▶ Name: 0, dtype: object

왜 쓰나 한 행은 그 시점에 **ぜん 모든 센서값**이다. 한 줄을 90도 돌려 세운 모습으로 나오므로, 특정 샷에서 무슨 일이 있었는지 한눈에 확인할 수 있다.

응용 여기서 **0**은 **위치(첫 번째)**가 아니라 **이름표**다. 지금은 인덱스가 순서대로라 결과가 같아 보이지만 의미는 어디까지나 이름표다. 인덱스를 글자로 바꾸면 그 차이가 분명해진다.

```
# 품질등급을 인덱스로 지정
df3 = pd.read_csv("data/13_diecasting_small.csv", index_col="품질등급")

print(df3.loc["불량"].shape)      # 라벨이 불량인 행 전부
print(df3.loc["불량"][["샷", "형체력"]].head(3))

# 이걸.iloc으로는 불가능하다 (정수만 받음)
# df3.iloc["불량"] ← TypeError
```

```
▶ (6, 6)
▶   샷   형체력
▶ 품질등급
▶ 불량   5   222.0
▶ 불량  10   247.0
▶ 불량  15   258.0
```

자주 하는 실수

`df.loc(0)`처럼 **소괄호**를 쓰면 오류다. `loc`은 인덱서라서 대괄호를 쓴다.

6-2. loc 범위 선택 — 끝 라벨까지 포함

패턴 `df.loc[시작:끝]` 라벨 범위로 여러 행 선택

정의 콜론으로 범위를 지정한다. **loc은 끝에 적은 라벨까지 포함한다** — 파이썬 기본 슬라이싱과 정반대다.

형태 `df.loc[0:4]`

```
print(len(df.loc[0:2]))    # 0, 1, 2 → 3줄
print(len(df.loc[0:4]))    # 0 ~ 4 → 5줄
print(len(df.loc[0:10]))   # 0 ~ 10 → 11줄

print(df.loc[0:2, ["샷", "형체력"]])
```

```
▶ 3
▶ 5
▶ 11
▶ 샷   형체력
▶ 0    1   258.0
▶ 1    2   257.0
▶ 2    3   253.0
```


왜 쓰나 "부른 이름까지 다 온다"가 loc의 규칙이다. 라벨은 이름이지 개수가 아니므로, 0번부터 10번까지 부르면 11개가 오는 것이 자연스럽다.

응용 시작이나 끝을 생략할 수 있다. `df.loc[:5]`는 처음부터 5번까지, `df.loc[10:]`는 10번부터 끝까지다. 생략해도 끝 포함 규칙은 그대로다.

```
print("처음~5번 :", len(df.loc[:5]))
print("10번~끝 :", len(df.loc[10:]))
print("전체   :", len(df.loc[:]))

# 이상 의심 구간만 떼어내 집중 분석
구간 = df.loc[10:14, ["샷", "형체력", "품질등급"]]
print(구간)
```

```
▶ 처음~5번 : 6
▶ 10번~끝   : 20
▶ 전체     : 30
▶ 샷  형체력 품질등급
▶ 10  11   250.0   양품
▶ 11  12   258.0   양품
▶ 12  13   354.0   주의
▶ 13  14   258.0   양품
▶ 14  15   258.0   불량
```

주의

`df.loc[0:10]`은 10개가 아니라 11개다. 정확히 N개가 필요하면 `iloc`을 쓴다. 이 함정은 STEP 8에서 자세히 다룬다.

6-3. loc 행과 열 동시 선택 — loc의 진짜 힘

패턴 `df.loc[행, 열]` 사각형 영역만 정확히 추출

정의 심표 앞은 행, 심표 뒤는 열. **행 먼저, 열 나중** — `shape`과 같은 순서다.

형태 `df.loc[0:2, ['품질등급', '형체력']]`

```

부분 = df.loc[0:2, ["품질등급", "형체력"]]
print(부분)
print("shape:", 부분.shape)

# 행 자리에 콜론 하나면 모든 행
전체행 = df.loc[:, ["형체력", "실린더압력"]]
print("전체행 shape:", 전체행.shape)

```

```

▶ 품질등급 형체력
▶ 0 양품 258.0
▶ 1 양품 257.0
▶ 2 주의 253.0
▶ shape: (3, 2)
▶ 전체행 shape: (30, 2)

```

왜 쓰나 행과 열을 동시에 좁히면 **분석에 필요한 사각형 영역만** 남는다. 데이터가 클수록 이 한 줄의 가치가 커진다.

응용 값 하나만 꺼낼 때도 같은 문법이다. `df.loc[0, "품질등급"]`처럼 쓰면 행 0의 품질등급 값 하나가 나온다. STEP 1에서 `df.loc["mean", "오일온도"]`를 썼던 것이 정확히 이 형태다.

```

# 값 하나
print(df.loc[0, "품질등급"])
print(df.loc[12, "형체력"])

# 열 하나 + 행 범위 → Series
print(df.loc[0:3, "형체력"].tolist())

# describe 결과에도 그대로 적용
print("형체력 평균:", round(df.describe().loc["mean", "형체력"], 1))

```

```

▶ 양품
▶ 354.0
▶ [258.0, 257.0, 253.0, 254.0]
▶ 형체력 평균: 267.8

```

개념

외우는 법 — **"줄, 즉 행이 먼저"**. 표를 읽을 때도 "몇 번째 줄의 어느 칸"이라고 말한다. 순서를 헷갈리면 엉뚱한 데이터를 분석하게 된다.

STEP 7

iloc

순서로 고르기 — 끝 직전까지만

7-1. iloc 위치 기반 선택

패턴 `df.iloc[위치]` 맨 위부터 센 순서로 행 선택

정의 인덱스가 무엇이든 상관없이 **위에서부터 센 순서 번호**로 행을 고른다. `i`는 integer의 `i`.

형태 `df.iloc[0]` · `df.iloc[0:5]` · `df.iloc[-1]`

```
print(df.iloc[0]["품질등급"])    # 맨 위 첫 번째 행
print(df.iloc[-1]["품질등급"])   # 맨 마지막 행
print(len(df.iloc[0:5]))         # 0~4 → 5줄
print(len(df.iloc[-3:]))         # 마지막 3줄
```

- ▶ 양품
- ▶ 불량
- ▶ 5
- ▶ 3

왜 쓰나 정렬 후 상위 `N`개를 뽑을 때 필수다. 인덱스가 뒤죽박죽이어도 "맨 위 5개"는 항상 `iloc[0:5]`다.

응용 음수 위치를 쓸 수 있는 것이 `iloc`의 강점이다. `iloc[-1]`은 마지막 행, `iloc[-3:]`은 마지막 3줄이다. 가장 최근 측정 기록을 볼 때 편리하다 — 순서로 세기 때문에 가능한 기능이다.

```
# 형체력이 가장 큰 상위 3개 샷
top3 = df.sort_values("형체력", ascending=False).iloc[0:3]
print(top3[["샷", "형체력", "품질등급"]])
```

```
# 가장 최근 2개 샷
print("\n최근 2개:")
print(df.iloc[-2:][["샷", "형체력"]])
```

```
▶ 샷 형체력 품질등급
▶ 22 23 359.0 주의
▶ 17 18 357.0 주의
▶ 27 28 355.0 주의
▶
▶ 최근 2개:
▶ 샷 형체력
▶ 28 29 255.0
▶ 29 30 253.0
```

자주 하는 실수

iloc의 두 가지 제약 — ① 정수만 받는다. df.iloc["양품"]은 오류. ② 조건 필터링이 안 된다. 조건은 loc이나 대괄호로 건다.

7-2. iloc 행과 열 동시 선택

패턴 df.iloc[행번호, 열번호] 행도 열도 순서 번호로

정의 loc과 같은 쉽표 구조인데, 이름 대신 0부터 시작하는 순서 번호를 쓴다. 열도 0부터 센다.

형태 df.iloc[0:2, 1:3]

```
print(df.columns.tolist())
print()
print(df.iloc[0:2, 1:3]) # 0~1번 행, 1~2번 열
```

```
▶ ['샷', '실린더압력', '주조압력', '사이클타임', '비스킷두께', '형체력', '품질등급']
▶ 실린더압력 주조압력
▶ 0 214.0 1037.0
▶ 1 217.0 1052.0
```

왜 쓰나 열 이름이 길거나 한글이라 타이핑이 번거로울 때, 또는 "앞쪽 6개 열" 처럼 위치로 지정하는 편이 자연스러울 때 쓴다.

응용 `iloc`은 행도 열도 끝을 포함하지 않는다. `1:3`은 1번과 2번 열만이다. 헷갈리면 `columns`로 열 순서를 미리 확인하자. 아래는 열 번호와 이름을 나란히 찍어 보는 방법이다.

```
for i, c in enumerate(df.columns):
    print(f" {i}번 → {c}")

print()
print("iloc[0:2, 1:3] 이 가져오는 열:", df.columns[1:3].tolist())
print("iloc[50:60, 0:6].shape:", df.iloc[0:10, 0:6].shape)
```

- ▶ 0번 → 샷
- ▶ 1번 → 실린더압력
- ▶ 2번 → 주조압력
- ▶ 3번 → 사이클타임
- ▶ 4번 → 비스킷두께
- ▶ 5번 → 형체력
- ▶ 6번 → 품질등급
- ▶
- ▶ `iloc[0:2, 1:3]` 이 가져오는 열: ['실린더압력', '주조압력']
- ▶ `iloc[0:10, 0:6].shape`: (10, 6)

주의

같은 영역을 `loc`은 이름으로, `iloc`은 번호로 짚는다. 두 가지 길이 있는 것이지 어느 하나가 우월한 게 아니다.

7-3. `iloc` 실전 — 정렬 후 상위·하위 리포트

`iloc`이 가장 빛나는 자리가 정렬 후 상위 N개다. 정렬하면 인덱스가 뒤섞이므로 라벨은 쓸 수 없고, 순서로 세는 `iloc`만이 정확히 N개를 준다.

`iloc`으로 만드는 상위 N개 리포트

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

def top_bottom(df, col, n=5):
    # 지표 기준 상위·하위 n개 리포트
    s = df.sort_values(col, ascending=False)
    print(f"[{col} 상위 {n}]\n")
    print(s.iloc[0:n][["샷", col, "품질등급"]].to_string(index=False))
    print(f"상위 {n} 평균 {s.iloc[0:n][col].mean():.1f} · 전체 평균 {df[col].mean():.1f}")

top_bottom(df, "형체력", 5)
```

```
▶ [형체력 상위 5]
▶ 샷 형체력 품질등급
▶ 23 359.0    주의
▶ 18 357.0    주의
▶ 28 355.0    주의
▶ 13 354.0    주의
▶ 20 259.0    불량
▶ 상위 5 평균 336.8 · 전체 평균 267.8
```

설비 적용

형체력 상위 4개가 전부 **주의** 등급이다. 형체력이 350을 넘는 샷은 정상이 아니라는 신호다 — 금형을 지나치게 세게 닫고 있다는 뜻일 수 있다. **정렬 한 번에 이런 패턴이 드러난다.** 다음 시간에 배울 `sort_values`와 조건 필터링을 결합하면 이런 리포트를 자동으로 뽑을 수 있다.

주의

하위 n개를 뽑으려면 `s.iloc[-n:]`를 쓴다. 내림차순 정렬했으니 **뒤쪽이 하위**다. `s.iloc[0:n]`과 `s.iloc[-n:]`을 짝으로 기억해 두면 상·하위 리포트를 한 함수로 만들 수 있다.

STEP 8

loc vs iloc

오늘 최다 실수 구간 — 끝 포함과 끝 제외

교안이 "오늘 수업에서 가장 실수가 잦은 부분 — 집중" 이라고 붉게 표시한 곳이다. 같은 0:2를 적었는데 결과 줄 수가 다르다.

8-1. 같은 숫자, 다른 줄 수

끝 포함 여부가 만드는 차이

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

print("loc [0:2] →", len(df.loc[0:2]), "줄 (0, 1, 2)")
print("iloc[0:2] →", len(df.iloc[0:2]), "줄 (0, 1)")
print()
print("loc [0:4] →", len(df.loc[0:4]), "줄")
print("iloc[0:4] →", len(df.iloc[0:4]), "줄")
print()
print("loc [0:10] →", len(df.loc[0:10]), "줄 ← 10개가 아니라 11개")
print("iloc[0:10] →", len(df.iloc[0:10]), "줄")
```

- ▶ loc [0:2] → 3 줄 (0, 1, 2)
- ▶ iloc[0:2] → 2 줄 (0, 1)
- ▶
- ▶ loc [0:4] → 5 줄
- ▶ iloc[0:4] → 4 줄
- ▶
- ▶ loc [0:10] → 11 줄 ← 10개가 아니라 11개
- ▶ iloc[0:10] → 10 줄

코드	가져오는 행	줄 수	규칙
df.loc[0:2]	0, 1, 2번	3줄	끝 라벨 포함
df.iloc[0:2]	0, 1번	2줄	끝 미포함
df.loc[0:4]	0 ~ 4번	5줄	끝 라벨 포함
df.iloc[0:4]	0 ~ 3번	4줄	끝 미포함
df.loc[0:10]	0 ~ 10번	11줄	끝 라벨 포함 — 함정

코드	가져오는 행	줄 수	규칙
df.iloc[0:10]	0 ~ 9번	10줄	끝 미포함

개념

교안이 준 외우는 문장이 정확하다 — "**loc은 부른 이름까지 다 온다, iloc은 끝 직전까지만 온다.**" 라벨은 이름이라 부르면 오고, 위치는 **개수**를 세는 것이라 파이썬 기본 슬라이싱 규칙(끝 제외)을 따른다. 15일차에 배운 `arange` · 슬라이싱의 "시작 포함 끝 제외"가 `iloc`에는 그대로 적용되고, `loc`만 예외인 셈이다.

8-2. 왜 이 실수가 무서운가 — 오류가 안 난다

`df.loc[0:10]`을 "10개 뽑기"로 알고 썼다고 하자. **오류는 나지 않는다.** 코드는 멀쩡히 돌고, 11줄짜리 결과도 그럴듯하다. 평균을 내면 숫자가 나오고 그래프도 그려진다. **틀린 줄을 모른 채 보고서가 나간다.**

같은 의도, 전혀 다른 결과

```
# "상위 10개 샷의 평균 형체력" 을 구하려 했다
s = df.sort_values("형체력", ascending=False)

wrong = s.loc[0:10, "형체력"]      # 의도: 상위 10개
right = s.iloc[0:10]["형체력"]    # 실제 상위 10개

print("loc 결과:", len(wrong), "줄 · 평균", round(wrong.mean(), 1))
print("iloc 결과:", len(right), "줄 · 평균", round(right.mean(), 1))
```

▶ **loc 결과:** 23 줄 · 평균 255.7
▶ **iloc 결과:** 10 줄 · 평균 297.4

10줄을 기대했는데 **23줄**이 나왔다. 평균도 255.7 대 297.4로 42나 차이 난다. 정렬 후에는 인덱스가 뒤죽박죽(22, 17, 27, 12 ...)이 되는데, 이때 `loc[0:10]`은 **현재 순서에서 라벨 0이 있는 자리부터 라벨 10이 있는 자리까지**를 통째로 잘라 온다. 정렬이 아무 의미가 없어질 뿐 아니라 줄 수도 예측이 안 된다.

개념

`loc`의 슬라이싱은 인덱스가 **정렬되어 있을 때만** "0부터 10까지 11줄"로 동작한다. 정렬 · 필터링으로 인덱스가 뒤섞이면(비단조, non-monotonic) 결과를 예측하기 어려워진다. **정렬 뒤에는 무조건 `iloc` 이라고 외워 두는 편이 안전하다.**

의도	써야 할 도구	이유
정확히 N개	<code>iloc[0:N]</code>	개수를 세는 것이므로 위치 기준
이름표 범위	<code>loc[시작:끝]</code>	특정 라벨 구간을 부르는 것
정렬 후 상위 N	<code>sort_values().iloc[0:N]</code>	정렬하면 라벨이 뒤섞임
특정 숫 번호	<code>loc[숫번호]</code> (index_col 지정 시)	이름으로 부르는 것

8-3. loc과 iloc 비교표 — 한 장 요약

비교 항목		
기준	라벨 (이름표)	위치 (순서)
받는 값	인덱스 값 — 숫자·글자 모두	0부터 시작하는 정수만
글자 인덱스	가능	불가 — <code>TypeError</code>
슬라이싱 끝	포함	제외
음수 인덱스	불가	가능 — <code>iloc[-1]</code>
조건 필터	가능 (다음 시간)	불가
주 용도	특정 라벨·구간 · 조건 필터	정확히 N개 · 정렬 후 상위

설비 적용

교안 p54의 마지막 문장이 다음 시간을 예고한다 — "**loc은 조건 필터의 무대**" 다. `df.loc[df["형체력"] > 300]`처럼 조건을 넣을 수 있는 것은 `loc`뿐이다. `iloc`은 정수만 받으므로 조건을 못 넣는다. 그래서 실무에서는 `loc`을 훨씬 많이 쓰게 된다. 강사도 "**필터링 때문에 loc을 더 많이 쓸 것**"이라고 짚었다.

STEP 9

다음 시간 맛보기

Boolean Series — 조건으로 고르기

오늘 마지막 15분은 다음 시간 주제인 **조건 필터링**의 도입부였다. 강사가 강조한 한 문장이 중요하다 — **"판다스의 새로운 문법이 아니라 넘파이에 있던 기법을 그대로 가져와 쓰는 것."**

9-1. 위치 기준과 조건 기준

지금까지 배운 `loc`·`iloc`은 **위치를 알 때** 쓰는 도구다. 그런데 현장의 질문은 대부분 위치를 모른다.

구분		조건 필터링
언제 쓰나	위치나 이름을 알 때	위치는 몰라도 성질을 알 때
질문 형태	"3번 행을 보자"	"비스킷두께가 위험하게 높은 샷은?"
기준	표에서의 자리	데이터의 내용
현장 빈도	상대적으로 낮음	압도적으로 높음

개념

강사가 든 예가 명확하다 — **"3만 개 중에 2만 5800번째 것을 굳이 뽑아낼 이유가 있나?"** 실무에서 위치로 뽑는 경우는 "최신 5개"처럼 제한적이다. 훨씬 자주 쓰는 건 **"조건에 맞는 것만 따로 뽑기"** — 결측이 있는 행만, 특정 설비만, 특정 날짜 구간만.

9-2. Boolean Series — 조건의 결과

개념 Boolean Series 행마다 참·거짓이 담긴 시리즈

정의 열 전체에 조건을 걸면 원본과 같은 길이의 **참·거짓 시리즈**가 만들어진다. 어느 행이 조건에 맞는지 알려 주는 지도 역할.

형태 `df['비스킷두께'] >= 13`

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")
```

```
조건 = df["비스킷두께"] >= 13
print(type(조건).__name__)
print(조건.head())
print("참 개수:", 조건.sum())
```

```
▶ Series
▶ 0    False
▶ 1    False
▶ 2     True
▶ 3    False
▶ 4     True
▶ Name: 비스킷두께, dtype: bool
▶ 참 개수: 6
```

왜 쓰나 16일차 넘파이의 **불리언 배열과 정확히 같은 원리**다. Series는 곧 넘파이 1차원 배열이므로, 넘파이에서 되던 것이 그대로 된다. 새 문법이 아니라 이미 아는 것의 확장이다.

응용 `.sum()`으로 참 개수를, `.mean()`으로 참 비율을 구하는 것도 넘파이와 동일하다. 16일차 STEP 3에서 배운 그대로다.

```
조건 = df["비스킷두께"] >= 13
```

```
print(f"조건 만족: {조건.sum()}건 / 전체 {len(df)}건 ({조건.mean()*100:.1f}%")
```

```
# 여러 조건도 넘파이와 같은 규칙 — 괄호 + & |
```

```
위험 = (df["비스킷두께"] >= 13) | (df["형체력"] < 240)
```

```
print(f"위험 시점: {위험.sum()}건")
```

```
▶ 조건 만족: 6건 / 전체 30건 (20.0%)
```

```
▶ 위험 시점: 6건
```

개념

`and`·`or`가 안 되고 `&`·`|`를 괄호와 함께 써야 하는 것도 넘파이와 똑같다. 16일차 STEP 5에서 배운 규칙이 그대로 적용된다.

9-3. Boolean Series로 행 추출 — 체로 거르기

문법 `df[조건]` 조건이 참인 행만 추출

정의 앞에서 만든 참·거짓 목록을 DataFrame 대괄호에 넣으면 **참인 행만 남는다**.

형태 `df[df['비스킷두께'] >= 13]`

```
result = df[df["비스킷두께"] >= 13]
print(result[["샷", "비스킷두께", "형체력", "품질등급"]])
```

```
▶ 샷 비스킷두께 형체력 품질등급
▶ 2      3    21.0  253.0   주의
▶ 4      5    14.0  222.0   불량
▶ 12     13    19.0  354.0   주의
▶ ...      (총 6줄)
```

왜 쓰나 체로 거르듯 참인 행만 통과시킨다. 결과도 다시 DataFrame이라 통계를 이어붙일 수 있다.

응용 두 단계로 나눠 생각하면 헛갈리지 않는다. ① **안쪽이 참·거짓 목록을 만들고** ② **바깥 대괄호가 참인 행만 꺼낸다**. 결과에 바로 통계를 이어붙일 수 있다.

```
위험 = df[df["비스킷두께"] >= 13]

print("위험 샷 평균 형체력:", round(위험["형체력"].mean(), 1))
print("전체 평균 형체력 :", round(df["형체력"].mean(), 1))
print("품질등급 분포:", 위험["품질등급"].value_counts().to_dict())
```

```
▶ 위험 샷 평균 형체력: 316.7
▶ 전체 평균 형체력 : 267.8
▶ 품질등급 분포:   주의 5 · 불량 1
```

설비 적용

비스킷두께 13 이상인 6개 샷 중 **5개가 주의, 1개가 불량** — 양품이 하나도 없다. 조건 하나 걸었을 뿐인데 벌써 분석다운 결론이 나온다.

연결

오늘 여기서 멈췄다. 다음 시간에는 `isin`(여러 값 매칭) · `between`(범위 조건) · `str.contains`(문자열 포함) · `sort_values`(정렬) · `copy`와 `SettingWithCopyWarning`까지 나간다. **오늘 배운 열 선택과 loc이 그 전부의 바탕이다**.

치트시트

오늘 나온 전부 — 한 장 요약

A-1. 탐색 도구 (12_02)

도구	형태	하는 일	기억할 점
<code>.head(n)</code> · <code>.tail(n)</code>	메서드	앞·뒤 n줄	짜으로 쓴다 · 큰 수 넣어도 오류 없음
<code>.shape</code>	속성	(행, 열)	괄호 없음
<code>.columns</code>	속성	열 이름 목록	<code>.tolist()</code> 로 리스트화
<code>.index</code>	속성	행 이름표 범위	0부터 시작
<code>.dtypes</code>	속성	열별 자료형	<code>int64</code> · <code>float64</code> · <code>object</code>
<code>.info()</code>	메서드	구조 + 결측 종합	Non-Null Count가 핵심
<code>.describe()</code>	메서드	숫자 열 8개 통계	글자 열은 자동 제외

A-2. describe 8개 통계량

통계량	뜻	읽는 요령
count	빈 칸 아닌 값의 개수	전체 행 수보다 작으면 결측
mean	평균	중앙값과 비교해 치우침 판정
std	표준편차	평균의 몇 %인지로 안정성 판단
min · max	최솟값 · 최댓값	양 끝 — 정상 범위 감각
25% · 75%	아래·위 4분의 1 지점	가운데 절반이 어디 모였나
50%	중앙값	평균과 비슷하면 고른 분포

A-3. 열 선택 (13_01 PART 01)

코드	결과	shape 예	언제
<code>df['형체력']</code>	Series 1차원	(30,)	통계 계산 · 조건 걸기

코드	결과	shape 예	언제
<code>df[['형체력']]</code>	DataFrame	(30, 1)	표 형태 유지
<code>df[['형체력', '실린더압력']]</code>	DataFrame	(30, 2)	여러 센서 함께
<code>df[['실린더압력', '형체력']]</code>	DataFrame	(30, 2)	적은 순서대로 열 배치
<code>df.columns.tolist()</code>	list	—	오타 방지 · 번호로 접근

A-4. 행 선택 (13_01 PART 02)

코드	뜻	결과
<code>df.loc[0]</code>	라벨 0인 행 전체	Series
<code>df.loc[0:2]</code>	라벨 0~2 (끝 포함)	DataFrame 3줄
<code>df.loc[0:2, ['품질등급', '형체력']]</code>	행 범위 + 열 이름	DataFrame (3, 2)
<code>df.loc[:, ['형체력']]</code>	모든 행 + 특정 열	DataFrame
<code>df.loc[0, '품질등급']</code>	행·열 지정 — 값 하나	스칼라
<code>df.iloc[0]</code>	위치 0번 행	Series
<code>df.iloc[0:2]</code>	위치 0~1 (끝 제외)	DataFrame 2줄
<code>df.iloc[-1]</code>	맨 마지막 행	Series
<code>df.iloc[-3:]</code>	마지막 3줄	DataFrame
<code>df.iloc[0:2, 1:3]</code>	행 0~1, 열 1~2 (둘 다 끝 제외)	DataFrame (2, 2)

A-5. loc vs iloc 한 줄 요약

핵심 문장	뜻
이름표로 고를 땐 loc, 순서로 고를 땐 iloc	오늘 단 하나만 가져간다면 이것
loc은 부른 이름까지 다 옴	<code>loc[0:10]</code> 은 10개가 아니라 11개
iloc은 끝 직전까지만 옴	<code>iloc[0:10]</code> 은 정확히 10개
정렬 뒤에는 무조건 iloc	정렬하면 라벨과 위치가 갈라진다
조건 필터는 loc만 가능	다음 시간의 무대

A-6. 자주 쓰는 조합 패턴

복붙해서 쓰는 여섯 패턴

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")
```

① 새 데이터 첫 3종 세트

```
df.head(); df.info(); df.columns.tolist()
```

② 숫자 열만 통계

```
print(df.select_dtypes(include="number").describe().round(1))
```

③ 필요한 센서만 골라 통계

```
cols = ["실린더압력", "주조압력", "형체력"]
print(df[cols].mean().round(1).to_string())
```

④ 정렬 후 상위 N개 (iloc 필수)

```
print(df.sort_values("형체력", ascending=False).iloc[0:3][["샷", "형체력"]])
```

⑤ 특정 구간 + 특정 열만

```
print(df.loc[10:14, ["샷", "형체력", "품질등급"]])
```

⑥ 값 하나 짚어내기

```
print(df.loc[12, "형체력"])
```

▶ 실린더압력 219.7

▶ 주조압력 971.5

▶ 형체력 267.8

▶ 샷 형체력

▶ 22 23 359.0

▶ 17 18 357.0

▶ 27 28 355.0

▶ 354.0

오류·함정 사전

✗ 이렇게 하면 → 무슨 일이 → ✓ 바로잡기

B-1. 오늘 만나기 쉬운 오류 10가지

✗ 이렇게 하면	무슨 일이 일어나	✓ 바로잡기
<code>df.loc(0)</code>	<code>TypeError: not callable</code>	<code>df.loc[0]</code> 인덱서는 대괄호
<code>df.shape()</code>	<code>TypeError: tuple not callable</code>	<code>df.shape</code> 속성은 괄호 없음
<code>df[형체력]</code> (따옴표 없이)	<code>NameError: 형체력 is not defined</code>	<code>df['형체력']</code>
<code>df['형체력', '실린더압력']</code>	<code>KeyError — 튜플로 해석</code>	<code>df[['형체력', '실린더압력']]</code> 두 겹
<code>df['형채력']</code> (오타)	<code>KeyError: 형채력</code>	<code>df.columns</code> 에서 복사해 붙이기
<code>df[0]</code> (행을 고르려고)	<code>KeyError: 0 — '0'이라는 열을 찾음</code>	<code>df.loc[0]</code> 또는 <code>df.iloc[0]</code>
<code>df.loc[0:10]</code> 을 10개로 착각	오류 없이 11줄 — 가장 위험	<code>df.iloc[0:10]</code> 정확히 10개
정렬 후 <code>df.loc[0:9]</code>	라벨 기준이라 상위 10개가 아님	<code>sort_values().iloc[0:10]</code>
<code>df.iloc['양품']</code>	<code>TypeError: 정수만 가능</code>	<code>df.loc['양품']</code> (index 지정 시)
<code>df.loc[0:2, '품질등급', '형체력']</code>	<code>IndexingError — 열도 리스트로</code>	<code>df.loc[0:2, ['품질등급', '형체력']]</code>

B-2. 제출 코드 교정 — 실제로 실행해 확인한 것

Practice/12_02_example.py와 Practice/13_01_example.py를 돌려 보고 정리했다. 오류로 멈추는 코드는 없었다. 다만 교안이 요구한 것과 다른 것을 한 부분이 몇 군데 있다.

① 실습 2 — usecols로 우회해서 핵심을 비껴갔다

교안 실습 2의 목표는 "대괄호 한 겹으로 Series, 두 겹으로 DataFrame"을 손으로 확인하는 것이다. 제출 코드는 read_csv의 usecols 옵션으로 열을 골라 왔다. 결과는 비슷하지만 오늘의 핵심인 대괄호 겹수를 전혀 쓰지 않았다.

Practice/13_01_example.py 실습 2

```
# x 제출 코드 — read_csv 단계에서 열을 걸렀다
df_2_1 = pd.read_csv("Data/13_diecasting_small.csv", usecols=["주조압력"])
["주조압력"]
df_2_2 = pd.read_csv("Data/13_diecasting_small.csv",
usecols=["실린더압력", "주조압력", "사이클타임"])

# ✓ 요구사항 — 이미 불러온 DataFrame에서 대괄호로 고른다
df = pd.read_csv("Data/13_diecasting_small.csv")

s = df["형체력"]                                # 한 겹 → Series
d = df[["형체력", "실린더압력"]]                # 두 겹 → DataFrame

print(type(s).__name__, s.shape)
print(type(d).__name__, d.shape)
print("형체력 평균:", round(s.mean(), 1))

▶ Series (30,)
▶ DataFrame (30, 2)
▶ 형체력 평균: 267.8
```

주의

`usecols`는 **파일에서 읽을 때** 열을 고르는 옵션이고, 대괄호는 **이미 메모리에 올라온 DataFrame에서** 고르는 문법이다. 둘 다 유용하지만 쓰는 자리가 다르다. 큰 파일에서 메모리를 아끼려면 `usecols`, 분석 도중 필요한 열만 볼 때는 대괄호다. **오늘 실습의 주제는 후자였다.**

② 실습 3 — 코드가 비어 있다

파일을 읽기만 하고 열 선택이 없다. 교안 요구는 "한 센서 열을 Series로, 여러 feature 열을 DataFrame으로 선택해 형태 확인" 이다.

Practice/13_01_example.py 실습 3

```
# ✓ 실습 3 보충
df_shot = pd.read_csv("Data/13_diecasting_shot.csv")

s = df_shot["형체력"]
print(type(s).__name__, s.shape)

cols = df_shot[["실린더압력", "주조압력"]]
print(type(cols).__name__, cols.shape)
```

```
▶ Series (200,)
▶ DataFrame (200, 3)
```

③ 실습 6 — 교안의 (10, 5)를 좌표로 오해했다

여기가 오늘 가장 중요한 교정이다. 교안 실습 6의 예상 결과 (10, 5) · (11, 2) · (10, 6) 은 좌표가 아니라 shape 이다. 제출 코드는 이것을 "10행 5열 칸의 값"으로 읽고 df.loc[10, cols[5]]처럼 값 하나씩 뽑았다.

Practice/13_01_example.py 실습 6

```
# x 제출 코드 — (10, 5) 를 좌표로 해석
print(f"cols[5]: {cols[5]} | {df_6.loc[10, cols[5]]}") # 값 하나

# ✓ 교안 의도 — shape 을 확인하는 문제
df_shot = pd.read_csv("Data/13_diecasting_shot.csv")

cols = ["실린더압력", "주조압력", "사이클타임", "비스킷두께", "형체력"]
print(df_shot[cols].iloc[0:10].shape) # (10, 5)
print(df_shot.loc[0:10, ["실린더압력", "주조압력"]].shape) # (11, 2)
print(df_shot.iloc[50:60, 0:6].shape) # (10, 6)
```

```
▶ (10, 5)
▶ (11, 2)
▶ (10, 6)
```

개념

(11, 2)가 결정적인 힌트다. loc[0:10]이 11줄이라는 것 — 오늘 배운 「loc은 끝 포함」을 결과로 확인시키려고 교안이 일부러 넣은 숫자다. 세 개가 나란히 (10, ...), (11, ...), (10, ...)인 이유가 여기 있다. 가운데만 11인 것을 보고 "아, loc이라서 하나 더구나"를 알아채는 것이 이 문제의 목적이다.

④ 실습 4·5 — 지정된 파일이 다르다

실습	교안 지정 파일	제출 코드	영향
실습 4	13_diecasting_small.csv 30행	13_diecasting_shot.csv 200행	loc/iloc 동작 확인은 되지만 예상 결과와 불일치
실습 5	13_diecasting_small.csv	13_diecasting_shot.csv	(5,2)·(5,3) 확인 못 함

그리고 실습 5는 세 단계 중 첫 단계만 했다. 교안은 ① loc로 행 범위+열 이름 ② 다른 행 범위에서 세 열 ③ iloc 음수 인덱스로 마지막 행 — 세 가지를 요구한다. 보충답안은 부록 C에 있다.

B-3. 원자료 오류 — 강사 코드와 교안

① 강사 코드 오타 — 실행이 멈춘다

13_combined.py 실습 5에 열 이름 오타가 있어 KeyError로 프로그램이 중단된다. 그대로 복사해 실행하면 여기서 멈춘다.

주조입력 → 주조압력

```
# x 강사 배포 코드 (13_06_practice_46_260813_6.py)
df_sub2 = df.loc[10:14, ["형체력", "실린더압력", "주조입력"]]
#
#                                     ↑ 주조입력

# KeyError: "['주조입력'] not in index"

# ✓ 올바른 열 이름
df_sub2 = df.loc[10:14, ["형체력", "실린더압력", "주조압력"]]
print(df_sub2.shape)

▶ (5, 3)
```

설비 적용

압력(壓力)과 입력(入力)은 한 글자 차이다. 한글 열 이름을 쓰면 이런 오타가 자주 난다. 예방책은 하나뿐이다 — df.columns.tolist()를 찍고 출력된 이름을 복사해서 붙여 쓰는 것. 강사도 수업 중 이 방법을 권했다.

② 교안 이탈자 — 실습 3 예상 결과

교안 (100, 3) → 실제 (200, 3)

교안 13_01 p24 실습 3 예상 결과: "한 열은 Series, 세 열은 (100, 3) 출력"

```
df_shot = pd.read_csv("data/13_diecasting_shot.csv")
print(df_shot.shape)
print(df_shot[["형체력", "실린더압력", "주조압력"]].shape)
```

- ▶ (200, 7)
- ▶ (200, 3) ← 교안은 (100, 3) 이라고 적혀 있다

주의

13_diecasting_shot.csv는 **헤더 1줄 + 데이터 200줄**이다. 교안이 100행짜리 파일을 기준으로 쓰였다가 데이터가 바뀐 것으로 보인다. **이럴 때는 실행 결과가 정답**이다. 슬라이드 숫자와 다르다고 자기 코드를 의심하지 말고, shape을 직접 찍어 확인하자.

③ 정리 — 원자료 오류 세 건

출처	내용	확인 방법
13_combined.py 실습 5	주조입력 → 주조압력	실행하면 KeyError로 멈춤
교안 13_01 p24	(100, 3) → (200, 3)	df_shot.shape 찍어 보면 (200, 7)
12_combined.py	오늘분(describe) 코드 없음 — 어제 파일	파일명 접미사가 42_260812

APPENDIX C

실습 하이라이트

제출한 것 · 빠진 것 · 다음 시간

C-1. 오늘 나온 실습 전체

실습	교안 요구	제출	한 줄 평
12_02 실습 6	describe로 이상 신호 찾기	제출	75% vs max 두 열 찾음 — 좋음
12_02 실습 7	통계량 문장으로 묘사	일부	"모터전류가 고르다고 생각하지 않는다" — 근거 보강 필요
12_02 실습 8·9	두 데이터 비교 · 첫 탐색 종합	미제출	아래 보충
13_01 실습 1	불러오기와 구조 확인	제출	head·columns·shape 모두 확인 — 좋음
13_01 실습 2	대괄호 한 겹·두 겹	우회	usecols로 대체 — 핵심 비껴감
13_01 실습 3	공정 센서 열 골라내기	비어 있음	아래 보충
13_01 실습 4	loc·iloc 행 선택	제출	끝 포함/제외 차이 주석까지 — 좋음
13_01 실습 5	행·열 동시 선택	일부	3단계 중 1단계만
13_01 실습 6	특정 구간 추출 종합	오해	(10,5)를 좌표로 읽음 — shape이 정답

C-2. 보충답안

13_01 실습 3 — 공정 센서 열 골라내기

실습 3 보충답안

```
import pandas as pd

df_shot = pd.read_csv("data/13_diecasting_shot.csv")
print("전체 shape:", df_shot.shape)

# 한 센서 열을 Series 로
s = df_shot["형체력"]
print(f"Series : {type(s).__name__} · shape {s.shape}")

# 여러 feature 열을 DataFrame 으로
features = df_shot[["형체력", "실린더압력", "주조압력"]]
print(f"DataFrame: {type(features).__name__} · shape {features.shape}")

print(features.head(3))
```

```
▶ 전체 shape: (200, 7)
▶ Series : Series · shape (200,)
▶ DataFrame: DataFrame · shape (200, 3)
▶ 형체력 실린더압력 주조압력
▶ 0  258.0  214.0  1037.0
▶ 1  257.0  217.0  1052.0
▶ 2  254.0  214.0  1037.0
```

13_01 실습 5 — loc·iloc 행·열 동시 선택 (3단계 전부)

실습 5 보충답안

```
df = pd.read_csv("data/13_diecasting_small.csv")
```

```
# ① loc 로 행 범위 + 열 이름
```

```
a = df.loc[0:4, ["품질등급", "형체력"]]
```

```
print("① shape:", a.shape)
```

```
# ② 다른 행 범위에서 세 열
```

```
b = df.loc[10:14, ["형체력", "실린더압력", "주조압력"]]
```

```
print("② shape:", b.shape)
```

```
# ③ iloc 음수 인덱스로 마지막 3행
```

```
c = df.iloc[-3:]
```

```
print("③ 줄 수:", len(c))
```

```
print(c[["샷", "형체력", "품질등급"]])
```

```
▶ ① shape: (5, 2)
```

```
▶ ② shape: (5, 3)
```

```
▶ ③ 줄 수: 3
```

```
▶ 샷 형체력 품질등급
```

```
▶ 27 28 355.0 주의
```

```
▶ 28 29 255.0 양품
```

```
▶ 29 30 253.0 불량
```

13_01 실습 6 — 특정 구간 추출 종합 (shape 확인 문제)

실습 6 보충답안 — 세 줄로 오늘 전부를 확인

```
df_shot = pd.read_csv("data/13_diecasting_shot.csv")
```

① 여러 feature 열 선택 후 iloc 로 앞 구간

```
cols = ["실린더압력", "주조압력", "사이클타임", "비스킷두께", "형체력"]
```

```
print("①", df_shot[cols].iloc[0:10].shape, " ← iloc 이라 10줄")
```

② loc 라벨 범위로 두 열 구간

```
print("②", df_shot.loc[0:10, ["실린더압력", "주조압력"]].shape, " ← loc 이라 11줄")
```

③ iloc 위치 범위로 앞쪽 열 구간

```
print("③", df_shot.iloc[50:60, 0:6].shape, " ← 행도 열도 끝 제외")
```

▶ ① (10, 5) ← iloc 이라 10줄

▶ ② (11, 2) ← loc 이라 11줄

▶ ③ (10, 6) ← 행도 열도 끝 제외

12_02 실습 8 — 두 데이터 비교

실습 8 보충답안

```
import pandas as pd
```

```
df_c = pd.read_csv("data/12_metro_compressor.csv")
```

```
df_d = pd.read_csv("data/12_metro_digital.csv")
```

```
for name, d in [("압축기", df_c), ("디지털 신호", df_d)]:
```

```
    na = d.isna().sum().sum()
```

```
    verdict = "바로 분석 가능" if na == 0 else "정리 필요"
```

```
    print(f"{name:8s} shape={str(d.shape):10s} 결측={na:3d}개 숫자열  
={len(d.describe().columns)}개 → {verdict}")
```

▶ 압축기 shape=(200, 7) 결측= 1개 숫자열=5개 → 정리 필요

▶ 디지털 신호 shape=(120, 4) 결측= 0개 숫자열=3개 → 바로 분석 가능

주의

교안은 "압축기는 깔끔, 디지털 신호는 지저분"이라고 설명하지만, **우리 폴더의 파일은 반대다** — 압축기에 결측이 1개 있고 디지털 신호에는 없다. 교안이 예시로 든 파일(300행 25열)과 배포된 파일(120행 4열)이 다르기 때문이다. **문서 설명보다 실행 결과를 믿자.** 다만 "현실 데이터는 대부분 지저분하다"는 교훈 자체는 그대로 유효하다.

C-3. 다음 시간 예고 — 조건 필터링과 정렬 (13_02)

오늘 마지막에 문을 연 주제가 다음 시간 본진이다. 교안 13_02는 54쪽인데 오늘은 p5~p12 정도만 훑었다.

주제	핵심 표현	오늘과의 연결
Boolean Series	<code>df['두개'] >= 13</code>	오늘 맛본 부분 — 넘파이 불리언 배열과 동일
조건으로 행 추출	<code>df[df['두개'] >= 13]</code>	오늘 맛본 부분
복합 조건	<code>(조건1) & (조건2)</code>	16일차 넘파이와 같은 규칙 — 괄호 필수
부정 조건	<code>~(조건)</code>	not이 안 되는 이유도 동일
여러 값 매칭	<code>df['등급'].isin(['주의', '불량'])</code>	새 도구
범위 조건	<code>df['형체력'].between(240, 300)</code>	새 도구
문자열 조건	<code>df['등급'].str.contains('불')</code>	새 도구
정렬	<code>df.sort_values("형체력", ascending=False)</code>	오늘 STEP 5에서 미리 씀
상위 N개	<code>정렬.head(5) .iloc[0:5]</code>	오늘 배운 iloc이 여기서 쓰임
.copy()와 경고	SettingWithCopyWarning	필터링 결과를 저장할 때

연결

강사가 정리한 사고 흐름이 다음 시간의 뼈대다 — **선택 → 필터 → 정렬**. "데이터를 CSV로 읽어 describe 한 번 돌리면 끝인 줄 알았는데 그게 아니구나. 그 사이사이에 프로세스가 더 있구나" 라는 말이 정확하다. **필요 없는 것을 걷어내고, 순서를 바꾸고, 필요한 것만 살린 다음에야 의미 있는 통계가 나온다.**

주의

강사가 마지막에 언급한 과제 안내 — **13_01 실습 6개를 정리해서 제출**하는 건이 사이트에 올라와 있고, 「**종합 실습 판다스 3**」 도 오늘 날짜로 추가 배포되었다. 부록 C-2의 보충답안을 참고해 실습 3·5·6을 채워 제출하면 된다.

CLOSING

맺음말

오늘의 성취와 복습 루틴

오늘 넘은 선 — 탐색에서 선택으로

오늘 오전에 `describe()`로 데이터 첫 탐색 4단계를 완성했다. `head` → `shape` → `info` → `describe`. 이 네 줄이면 어떤 데이터를 받아도 "무엇에 관한 것이고, 얼마나 크고, 어디가 문제이고, 값이 정상인가"에 답할 수 있다.

오후에는 선택을 배웠다. 대괄호 겹수로 Series와 DataFrame이 갈리고, `loc`과 `iloc`으로 라벨과 위치가 갈린다. 그리고 마지막에 다음 시간의 무기인 조건 필터링을 살짝 봤다. "전체가 아니라 필요한 부분만 정확히 꺼내는 것" — 이것이 오늘의 한 문장이다.

오늘 할 수 있게 된 것	확인 질문	못 하면
<code>describe</code> 로 통계를 한 번에 뽑는다	<code>count</code> 가 전체 행 수보다 작으면 무슨 뜻인가?	STEP 1
통계로 이상 신호를 잡는다	75%와 <code>max</code> 가 멀면 무엇을 의심하나?	STEP 2
첫 탐색 4단계를 순서대로 한다	네 단계와 각각의 질문을 말할 수 있나?	STEP 3
대괄호 겹수를 구분한다	<code>df[['A']]</code> 의 결과는 Series인가 DataFrame인가?	STEP 4
라벨과 위치를 구분한다	정렬하면 왜 둘이 갈라지나?	STEP 5
<code>loc</code> 으로 행·열을 동시에 고른다	<code>df.loc[0:2, [2개 열]]</code> 의 shape은?	STEP 6
<code>iloc</code> 으로 상위 N개를 뽑는다	<code>iloc</code> 이 정수만 받는 이유는?	STEP 7
끝 포함/제외를 의식한다	<code>loc[0:10]</code> 은 몇 줄인가?	STEP 8
조건이 Boolean Series라는 걸 안다	넘파이의 무엇과 같은 원리인가?	STEP 9

복습 루틴 — 40분

오늘은 문법 자체는 어렵지 않은데 구분해야 할 것이 많다. 한 겹/두 겹, 라벨/위치, 끝 포함/제외. 셋 다 "둘 중 어느 쪽인가"를 묻는 문제라 직접 쳐서 결과를 눈으로 봐야 붙는다.



단계	무엇을	왜
① 부록 A 훑기 (5분)	치트시트 A-3·A-4·A-5를 눈으로 넘긴다	오늘 나온 표기법 전체를 다시 배치
② 보충답안 치기 (15분)	C-2의 실습 3·5·6을 보지 않고 쳐 본다	실습 6은 오늘 전부를 세 줄로 확인시킨다
③ 세 갈림길 손으로 (10분)	한겹/두겹 · 라벨/위치 · 끝포함/제외를 <code>type()</code> · <code>len()</code> 으로 확인	셋 다 눈으로 봐야 붙는다
④ 부록 B 소리내어 (10분)	$x \rightarrow \checkmark$ 를 입으로 말하며 넘긴다	<code>loc[0:10]</code> 은 반드시 한 번은 틀린다

세 갈림길을 한 파일로

③ 세 갈림길을 10분 만에 확인하는 코드

```
import pandas as pd
df = pd.read_csv("data/13_diecasting_small.csv")

print("[갈림길 1] 대괄호 겹수")
print(" 한 겹:", type(df["형체력"]).__name__, df["형체력"].shape)
print(" 두 겹:", type(df[["형체력"]]).__name__, df[["형체력"]].shape)

print("\n[갈림길 2] 라벨 vs 위치")
s = df.sort_values("형체력", ascending=False)
print(" loc[0] :", s.loc[0, "형체력"], " (라벨 0번)")
print(" iloc[0]:", s.iloc[0]["형체력"], " (맨 위)")

print("\n[갈림길 3] 끝 포함 vs 제외")
print(" loc[0:2] :", len(df.loc[0:2]), "줄")
print(" iloc[0:2]:", len(df.iloc[0:2]), "줄")
```

-
- ▶ [갈림길 1] 대괄호 겹수
 - ▶ 한 겹: Series (30,)
 - ▶ 두 겹: DataFrame (30, 1)
 - ▶
 - ▶ [갈림길 2] 라벨 vs 위치
 - ▶ loc[0] : 258.0 (라벨 0번)
 - ▶ iloc[0]: 359.0 (맨 위)
 - ▶
 - ▶ [갈림길 3] 끝 포함 vs 제외
 - ▶ loc[0:2] : 3 줄

▶ `iloc[0:2]`: 2 줄

설비 적용

취업 관점에서 오늘 내용은 **면접에서 가장 자주 나오는 Pandas 질문 두 개**를 포함한다. ① `loc`과 `iloc`의 차이는?
② `df["A"]`와 `df[["A"]]`의 차이는? 둘 다 "끝 포함 여부"와 "Series vs DataFrame"이라는 명확한 답이 있다. 오늘 배운 것을 그대로 말하면 된다. 그리고 부록 A-6의 여섯 패턴을 GitHub 레포에 정리해 두면 실무 감각을 보여 주는 자료가 된다.

다음 시간에는 조건으로 데이터를 고르고 정렬한다. 위치를 몰라도 성질만 알면 원하는 데이터를 다 찾아내는 능력 — 현장의 진짜 질문에 답하는 도구다. 오늘 배운 열 선택과 `loc`이 그 전부의 바탕이 된다.

— 끝 —